

WID Cryptography Specification

Version: 1.0.0

Status: Active

Scope

This document specifies cryptographic capabilities for **wid** CLIs, initially focusing on digital signatures for integrity and authenticity. The goal is to provide a robust yet energy-efficient mechanism for attesting to the origin and integrity of WIDs and associated data, particularly for resource-constrained IoT and agent systems.

Design Goals

1. **Integrity & Authenticity:** Ensure WIDs and associated data have not been tampered with and originate from a trusted source.
2. **Energy Efficiency:** Utilize cryptographic algorithms and practices suitable for low-power, embedded, and IoT environments.
3. **Simplicity:** Provide a straightforward API for cryptographic operations within the **wid** CLI.
4. **Security:** Leverage well-vetted and standard cryptographic primitives.

Algorithms

Digital Signature Algorithm (DSA)

- **Algorithm: Ed25519 (Edwards-curve Digital Signature Algorithm)**
- **Rationale:** Ed25519 is chosen for its:
 - **High security:** Provides 128-bit security level.
 - **Efficiency:** Fast signing and verification, making it suitable for resource-constrained devices.
 - **Small key and signature sizes:** Private keys are 32 bytes, public keys are 32 bytes, and signatures are 64 bytes.
 - **Collision resistance:** Strong collision resistance, a key goal of WID.
 - **Simplicity:** Designed to be easy to implement correctly and securely.

Key Formats

Keys (private and public) for Ed25519 will be expected in standard **PEM** (Privacy-Enhanced Mail) format.

- **Private Key (Signing Key):** -----BEGIN ED25519 PRIVATE KEY-----
- **Public Key (Verification Key):** -----BEGIN PUBLIC KEY----- (or -----BEGIN ED25519 PUBLIC KEY----- if supported by the library)

Implementations should provide or utilize tooling for generating Ed25519 key pairs.

Signature Format

Digital signatures generated by `wid sign` will be **Base64 URL-safe encoded** for ease of transport and storage in text-based systems. The raw signature bytes (64 bytes for Ed25519) will be encoded.

Operations

1. `wid sign` (Action: `A=sign`)

Purpose: To generate a digital signature for a WID string and optionally additional data.

Canonical Invocation:

```
wid A=sign WID=<wid_string> KEY=<private_key_path> [DATA=<data_path>] [OUT=<output_path>]
```

Parameters:

- `WID=<wid_string>` (Required): The WID string to be signed.
- `KEY=<private_key_path>` (Required): Path to the Ed25519 private key file in PEM format.
- `DATA=<data_path>` (Optional): Path to a file containing additional binary or text data to be included in the signature calculation. If provided, the data from this file is appended to the WID string (after converting WID string to bytes) before hashing.
- `OUT=<output_path>` (Optional): Path to a file where the Base64 URL-safe encoded signature will be written. If not provided, the signature will be output to `stdout`.

Signature Generation Process:

1. The WID string is converted to UTF-8 bytes.
2. If `DATA` is provided, its content is read as raw bytes.
3. The combined byte sequence (WID bytes + DATA bytes) is used as the message for the Ed25519 signing algorithm.
4. The Ed25519 private key (from `KEY`) is used to generate the 64-byte signature.
5. The 64-byte signature is then Base64 URL-safe encoded.

Output: The Base64 URL-safe encoded signature string.

2. `wid verify` (Action: `A=verify`)

Purpose: To verify a digital signature against a WID string and optionally additional data using a public key.

Canonical Invocation:

`wid A=verify WID=<wid_string> KEY=<public_key_path> SIG=<signature_string> [DATA=<data_path>`

Parameters:

- **WID=<wid_string>** (Required): The WID string that was signed.
- **KEY=<public_key_path>** (Required): Path to the Ed25519 public key file in PEM format.
- **SIG=<signature_string>** (Required): The Base64 URL-safe encoded digital signature to be verified.
- **DATA=<data_path>** (Optional): Path to a file containing additional binary or text data that was included in the original signature calculation. If provided, the data from this file is appended to the WID string (after converting WID string to bytes) before hashing for verification.

Signature Verification Process:

1. The WID string is converted to UTF-8 bytes.
2. If DATA is provided, its content is read as raw bytes.
3. The combined byte sequence (WID bytes + DATA bytes) is used as the message for the Ed25519 verification algorithm.
4. The SIG string is Base64 URL-safe decoded to retrieve the 64-byte raw signature.
5. The Ed25519 public key (from KEY) and the raw signature are used to verify the message.

Output: * 0 (Success): If the signature is valid. * 1 (Failure): If the signature is invalid, the key is invalid, or any other verification error occurs. A descriptive error message will be printed to **stderr**.

3. wid w-otp (Action: A=w-otp)

Purpose: Generate and verify short OTP codes bound to a specific WID using a shared secret.

Canonical Invocation:

`wid A=w-otp MODE=gen KEY=<secret_or_path> [WID=<wid_string>] [DIGITS=<4..10>]`

`wid A=w-otp MODE=verify KEY=<secret_or_path> WID=<wid_string> CODE=<otp_code> [DIGITS=<4..10>]`

Parameters:

- **MODE** (Optional): **gen** (default) or **verify**.
- **KEY** (Required): Shared secret (inline) or path to a file containing the secret.
- **WID** (Required in verify): WID used as OTP input. In **gen**, if omitted, implementation may auto-generate a WID from canonical W/Z/T.
- **CODE** (Required in verify): OTP code to verify.
- **DIGITS** (Optional): OTP length, default 6, allowed range 4..10.
- **MAX_AGE_SEC** (Optional, verify mode): maximum accepted age for the WID timestamp; default 0 (disabled).

- **MAX_FUTURE_SEC** (Optional, verify mode): maximum accepted future clock skew; default 5.

Computation:

1. Compute `HMAC-SHA256(secret, wid_utf8)`.
2. Take the first 4 digest bytes as an unsigned 32-bit integer.
3. Compute `otp = value mod 10^DIGITS` and zero-pad to DIGITS.

Output: * **MODE=gen:** JSON payload with `wid`, `otp`, `digits`. * **MODE=verify:** success message + exit 0 if valid; invalid message + exit 1 otherwise. * When time window args are set, verification also rejects stale/future WID timestamps.

1.0.0 rollout scope: * Primary: `sh`, `rust`, `c`, `go`, `python`, `typescript`.

Conformance

Implementations **MUST** pass all test cases defined for cryptographic operations in `spec/conformance/crypto.json` to be considered compliant.

Signed event envelope conformance and security hardening:

- Envelope spec: `spec/SIGNED_ENVELOPE_SPEC.md`
- Envelope fixtures: `spec/conformance/signed_envelope.json`
- Envelope compatibility fixtures: `spec/conformance/signed_envelope_v1.json`, `spec/conformance/signed_envelope_v1_1.json`
- Security matrix: `spec/conformance/security_matrix.json`

Future Considerations

- **Symmetric Encryption/Decryption:** Adding `A=encrypt` and `A=decrypt` actions using a robust, efficient symmetric cipher (e.g., AES-GCM-256).
- **Key Generation Utility:** A `wid genkey` command to simplify Ed25519 key pair generation.
- **Key Derivation:** Mechanisms for deriving keys from WIDs or other deterministic inputs.
- **Transport Security:** Integrating cryptographic operations directly with `wid` transports (e.g., signing MQTT messages).